Task: build the analytics platform

The plan is deliberately incremental. Milestone 1 gets a working pipeline end to end using Postgres, which you already know. Milestone 2 swaps the storage engine underneath it. If milestone 2 goes badly, milestone 1 is still a working platform — that's the point of the order.

The tools

Tool	What it does	Arrives in	Container?
Airflow	Schedules and runs the pipeline. Retries failures, shows what ran.	M1	Existing
dbt	Defines the transformations as SQL files, plus tests on the output.	M1	No, `pip install`
Cosmos	Bridge. Turns each dbt model into its own Airflow task so you can see them individually.	M1	No, `pip install`
Superset	The dashboards. The only UI anyone outside the team touches.	M1	New
Redis	Caches Superset queries so dashboards don't re-query on every load.	M1	New
Postgres	M1: holds the source data and the vault.	M1	Existing

Tool	What it does	Arrives in	Container?
	M2: holds the source data and the lake catalog.		
DuckDB	The query engine. A library, not a server — it runs inside other processes.	M2	No, `pip install`
DuckLake	The table format. Ties the Postgres catalog to the Parquet in MinIO so they behave like real tables.	M2	No, an extension
MinIO	Object storage. Holds the silver layer as Parquet. Speaks the S3 API.	M2	New
API ingestion	Pulls source data from APIs instead of CSV drops.	M3	TBD

Step 0 — Repo

Create a GitHub repo before you write anything else. Commit regularly and in small pieces — one commit per working change, not one commit per day. Everything lives here: the compose file, Dockerfiles, the dbt project, the DAGs, Superset config.

Nobody should ever need a file that only exists on your laptop.

Milestone 1 — Airflow, dbt, Cosmos, Superset on Postgres

Get the orchestration working against the vault that already exists in Postgres. No DuckDB, no MinIO, nothing new in the storage layer.

- Compose brings up Airflow, Superset, Redis, and connects to the existing Postgres.
- Airflow image rebuilt with `dbt-postgres` and `astronomer-cosmos`.
- The dbt project models the existing silver vault, with tests on every model.
- Cosmos renders it so each model is its own Airflow task.
- Superset connects to Postgres and reads the vault. One dashboard, three charts, two filters.

How it flows: Airflow triggers the DAG → Cosmos runs each dbt model as a task → dbt sends SQL to Postgres → Postgres does the work → Superset reads the result from Postgres.

Milestone 2 — DuckLake and DuckDB

Swap the storage layer. The models stay; where they land changes.

- Add MinIO to compose. Add a `ducklake_catalog` database on Postgres.
- Airflow image gains `duckdb` and `dbt-duckdb`. Superset image gains `duckdb` and `duckdb-engine`.
- dbt profile switches from the Postgres adapter to the DuckDB adapter, attaching the source Postgres read-only and the lake for output.
- Silver now materialises as Parquet in MinIO instead of tables in Postgres.
- Superset repoints at the lake.

How it flows now, step by step:

1. Airflow triggers the DAG. Cosmos has already turned each model into a task.
2. A task starts on a worker and launches dbt — a normal Python process.
3. dbt opens a DuckDB session **in memory**, loads the `postgres`, `httpfs` and `ducklake` extensions, then attaches the source Postgres read-only and the lake.
4. dbt compiles the model into one plain SQL statement and hands it to DuckDB. **dbt never touches your data** — it writes SQL, that's all.
5. DuckDB pulls the source rows over the wire, runs the joins in memory, writes the result as Parquet into MinIO, and writes rows into the catalog recording which files make up that table. Those two writes are **one transaction**, arbitrated by Postgres — which is why several model tasks can run at once safely.
6. The session closes and DuckDB disappears. Nothing persists in DuckDB. Only the Parquet in MinIO and the catalog rows in Postgres.
7. Superset starts its *own* DuckDB, attaches the *same* lake, finds the current Parquet via the catalog, and draws the chart. Redis caches it.

Airflow and Superset never speak to each other. They meet in the lake.

Done when: the lake reproduces the Postgres vault row for row, **and** a dashboard loads correctly while a DAG run is in progress. That second one is the real test.

Milestone 3 — API ingestion

Replace the CSV drops with API pulls. Spec to follow — do not start this before M2 is green.

Broadly: a new Airflow DAG that calls the source APIs, handles pagination and auth, lands the results, and feeds the same models. The transformation layer shouldn't need to change.

Milestone 4 — Finalisation

Turn it from a demo into something someone else can run.

- Compaction and snapshot expiry on a schedule, or small Parquet files pile up and queries slow.
- Backups for `ducklake_catalog` — lose it and the Parquet is orphaned with no schema.
- Secrets out of the compose file and into environment variables.
- Superset roles and access control, since DuckLake itself has no permission system.
- Alerting when a DAG fails.
- A README and a runbook: how to add a model, what to check when a dashboard is slow.
- Retire the old Postgres silver tables.
